

APUNTES TEMA 3

30/10/2025 (Hoy no he ido puto bus)

PROGRAMACIÓN ORIENTADA A OBJETOS (POO)

¿Por qué cambiamos a POO? → El principal problema es que al aumentar el tamaño y complejidad de la programación se volvió más inviable por lo que alrededor de los 70 se empezó a imponer el estilo de programación POO.

¿En qué consiste el estilo de programación POO? → aquí un programa no es un código que llama a procedimientos, sino que el programa es un conjunto de objetos. Y a los objetos no les importan los *otros objetos* **sino** que se centran en los mensajes que son capaces de responder.

OBJETOS

Una característica clave de los objetos es que todos presentan atributos que lo describen. Esto hará que todos los objetos de la misma clase o tipo vengan definidos por los mismos atributos, no obstante cada objeto tendrá unos valores concretos (pensar analogía pelota).

Todos los objetos realizan comportamientos u operaciones que especifican lo que hacen, a esto lo llamaremos **métodos**.

Los objetos no solo contendrán datos, sino que también *funcionalidades para* manipular dichos objetos. Lo que permitirá una **entidad compacta y reutilizable**.

Paradigma Resumen: con este estilo de programación el programador tendrá que determinar los objetos, sus características comunes y las acciones que realizan.

COMPRENDER: la programación que hemos hecho hasta ahora no es la efectiva en el mundo laboral, sino que era para adquirir una base.

LAS CLASES EN JAVA

¿Qué es una clase? → Una clase podemos verlo como una plantilla que definirá la forma de un tipo de objeto. En ella especificaremos los **atributos y comportamientos** con los que va a contar los objetos de dicha plantilla.

instanciar un objeto → Crear un objeto a partir de una clase (los objetos son las instancias de las clases)

Comprensión Paradigma: Un programa POO es una colección estructurada de clases que definen los distintos tipos de objetos que van a intervenir en la resolución del problema.

COMPONENTES DE UNA CLASE:

1. **ATRIBUTOS O PROPIEDADES:** características y estructuras de almacenamiento que tendrá los objetos de la clase (los valores de cada características son independientes)
2. **MÉTODOS:** conjunto de funcionalidades que describen el comportamiento que tendrán los objetos de la clase.

EJEMPLO VISUAL:

Atributos	Clase Coche	Métodos	
	<pre>private String Marca private String Modelo private String Color private String Matricula private double Precio static double descuento = 2000€</pre>		<pre>Arrancar() Detenerse() Acelerar() Frenar() verPrecio() punto.com no es un lenguaje</pre>

DECLARACIÓN DE CLASES

ASPECTOS A TENER EN CUENTA:

1. **DEFINIR E IMPLEMENTACIÓN DE UNA CLASE JAVA EN EL MISMO ARCHIVO:**
esto nos dice que el código donde defines cómo funciona la clase (sus atributos, métodos, etc.) se escribe **en el mismo archivo .java** donde se declara.

Ejemplo: si defines una clase **Cuenta**, todo lo que describe a esa clase (sus variables, métodos, constructores...) debe estar dentro del mismo archivo **Cuenta.java**.

2. **MANTENER EL ORDEN CREANDO UNA SOLA CLASE POR ARCHIVO .JAVA**
3. **ARCHIVO JAVA DEBERÁ TENER EL MISMO NOMBRE QUE LA CLASE PUBLIC QUE LO CONTIENE**
4. **EL TOKEN “class” NOS PERMITIRÁ DEFINIR UNA CLASE**
5. **EN LA DEFINICIÓN DE CLASE INCLUIAMOS LOS ATRIBUTOS QUE CONTIENE Y LOS MÉTODOS:**

ATRIBUTOS: es una **variable** que pertenece a una clase y que describe algo que tiene cada objeto de esa clase.

```
public class Persona {
    String nombre;    // atributo
    int edad;         // atributo
    double altura;    // atributo
}
```

MÉTODOS: función o acción que pertenece a una clase.

```
// Método para ingresar dinero
public void ingresar(double cantidad) {
    saldo += cantidad; // modifica el atributo saldo
}
```

NOMENCLATURA: los nombres deben ser **sustantivos**, primera letra **mayúscula** y resto en minúscula. Deben contener palabras completas y evitar el uso de abreviaturas.

PARTES BASICAS CLASE

1. Cabecera de la clase
2. Cuerpo de la clase
3. Comentarios

CABECERA DE LA CLASE:

En esta sección de la clase hallaremos tanto el nombre de la clase como los nombres de modificadores de las clases.

MODIFICADORES DE ACCESO

“Es la posibilidad o no que se tiene de acceso a una zona específica del código”.

La palabra **“public”** nos indicará un accesos a la clase de tipo público. Lo que permitirá que **cualquier clase** pueda hacer uso de ella. Esto también quiere decir que puedes acceder a ellas desde cualquier paquete dentro del proyecto java.

Si **no** aparece la palabra *public* (ni **ningún modificador**) la clase no **será pública** y su nivel de acceso lo denominaremos **“default”**, lo que quiere decir que solo pueden relacionarse con ella las clases pertenecientes a su mismo paquete.

La palabra **“private”** sólo se podrá acceder (verlo, editarlo...) aquellos trozos que estén dentro de la clase.

RESUMEN:

1. public → **todos los paquetes**
2. default → **las clases dentro del paquete**
3. private → **solo los trozos de código dentro de la clase**

Recomendación de uso: a los **atributos** vamos a denominarlos “**private**” y a los **métodos** utilizaremos “**public**”.

¿Qué otros niveles de accesos existen? →

IMPORTANTE: En una clase no puede haber más de una clase pública.

¿Cómo se interpreta la notación utilizada? → los corchetes [] indican que lo que va adentro puede tanto aparecer como no hacerlo, sin embargo lo que no va entre corchetes, si sera obligatorio que aparezca.

MÉTODOS DE UNA CLASE

CABECERA DEL MÉTODO:

Estructura Cabecera: **public** + **devolución** + [**verbo**] + (**Parámetros**) {
 Cuerpo →
 }

En los “**Parámetros**” o también llamados argumentos, le indicaremos datos necesarios para realizar la acción. (“Si no son atributos”). Ahi

La “devolución” será el tipo de dato que me da el resultado del método. **Un ejemplo seria una suma de enteros me devolverá un entero pues ahí colocaremos un int.**
Si la “devolución” no devuelve nada colocaremos en ese espacio el **token “void”**.

RECOMENDACION → cuando la **devolucion** no es un “**void**” la ultima linea del metodo introduciremos la sintaxis “**return + nombre variable;**”

Para llevar el método de la clase hacia la clase Principal

→ (**nombreclase**) (**op**) = **new** (**nombreclase**) ();

IMPORTANTÍSIMO: Los métodos **NO** imprimen en pantalla, esto si estamos usando métodos operaciones, obviamente si estamos usando el void, está enfocado en imprimir ahi si.

- - En los return no se hacen asignaciones - -

03/11/2025

OBJETOS DE UNA CLASE

Forma de instanciar objeto → (Clase) + (N. variable) = new (Nom. Clase) (Atributos)
DECLARAR INSTANCIA

Tras realizar este proceso nos generará como resultado una “**dirección**”. A diferencia de las variables que podemos machacar los anteriores valores aquí no podemos reemplazarlos.

- - Cada instancia que creemos será un objeto - -

CÓMO DECLARAR UN OBJETO

```
areaCirculo c1;
```

COMO INSTANCIAMOS UN OBJETO

```
c1= new areaCirculo (radio);
```

→ Entre paréntesis añadiremos los valores de los atributos, el **error** que aparece se debe a que no tenemos constructor.

CONSTRUCTOR:

Será el encargado de construir el objeto, está definido en la clase circulo.

→ **Java** por defecto nos dará un constructor para aquellos valores vacíos, sin embargo si queremos añadir parámetros debemos tener en la clase del objeto un **constructor con parámetro**.

- a. **¿CÓMO SE CREAN?**: Para crear un constructor debemos ir a la clase del objeto, desde ahí escribiremos el bloque de sintaxis:

```
public areaCirculo (double radio) { //Radio es una variable  
}
```

→ Deberán llevar el **mismo** nombre que la clase.

→ La variable dentro de los paréntesis deberá tener el mismo nombre que la variable que declaramos en los atributos.

```
private double radio;
```

Explicación clase: diferenciaremos entre colores, el azul oscuro será la variable que recibirá el dato desde la main y luego desde ahí vamos a inicializarlo con el azul clarito, el cual es la variable.

```
this.radio=radio;
```

- - Los constructores solo guardan datos de las variables que o bien leen/ o se lo damos, y luego lo guardan en los atributos de la clase. No realizan operaciones - -

COMO SE IMPRIME EN PANTALLA

Primer Método: Escribirlo desde un void.

```
public void imprimirDatos () {
    System.out.println("El valor es: "+radio);
}
```

CLASE - OBJETO

```
c1.imprimirDatos();
```

CLASE - MAIN

El "c1.imprimirDatos();" sería una **llamada** a método.

COMPRENDER: No hay objetos en la clase sino que estos están en la clase **main**.

RECORDATORIO SOBRE USO DE LOS BLOQUES: si el atributo está declarado en la clase no habrá necesidad de incluirlo en los parámetros.

```
private double radio;
```

```
public double calcularArea () {
    //variables
    double area=0, dos=2;
    area=Math.PI*Math.pow(radio, dos);
    return area;
}
```

COMO HACER LLAMADAS A MÉTODOS:

Para no volver a escribir las mismas líneas:

```
//calcular area
public double calcularArea () {
    //variables
    double area=0, dos=2;
    area=Math.PI*Math.pow(radio, dos);
    return area;
}

// calcular el radio a metros cuadrados
public double conversionAreaEnMetro2 () {
    double area=0, dos=2, conversion=10000;
    radio=radio/conversion;
    area=calcularArea();
    return area;
}
```

CÓMO GESTIONAR LOS CONSTRUCTORES

Generalmente vamos a intentar crear siempre DOS constructores:

1. Constructor que contenga todos los parámetros que vayamos a contener en los atributos.
2. Constructor vacío para probar el programa.
3. Según el contexto iremos regulando hacia más casos.

- - En los constructores es esencial llevar un orden en los atributos, utilizar el orden de arriba hacia abajo que hay en la declaración de los atributos - -

```
//DECLARAR CONSTRUCTORES

//con todos los parametros
public Cabecera (String nomAsign, int aula, String fecha) {
    this.nomAsign=nomAsign;
    this.aula=aula;
    this.fecha=fecha;
}

// CONSTRUCTOR VACIO
public Cabecera () {
}

// CON DOS PARAMETROS
public Cabecera (String nomAsign, int aula) {
    this.nomAsign=nomAsign;
    this.aula=aula;
}
```

05/11/2025

```
// DECLARAR ATRIBUTOS
```

```
private String nomAsign; 1
private int aula; 2
private String fecha; 3
```

utilizar el mismo nombre de un método ya existente con los mismos parámetros o bien cambiar el orden (si

EXTRAER VALORES DE LOS ATRIBUTOS EN MAIN Y MODIFICARLOS DESDE MAIN

IMPORTANTE → No podemos eliminarles el estado **private** a los atributos de la clase para poder modificarlos desde otra clase.

¿CÓMO SE HACE ENTONCES?

Para ello crearemos **métodos públicos** a los cuales podremos acceder desde main para poder modificarlos.

A estos métodos los denominaremos **getters** and **setters**. Cada get y un set para **cada atributo**. (Si hay 10 atributos deberemos hacer 10 get y 10 set para cada atributo, en el caso de que nos lo pidan)

EN EL CÓDIGO → Los getters and setters **van a ir** debajo del bloque **constructor**.

Ejemplo básico:

```
// GETTERS AND SETTERS
//Instituto

public String getInstituto () { //No pasa nada y devuelve nada
    return instituto;
}

public void setInstituto (String instituto) { //No devuelve nada y le paso algo
    this.instituto=instituto; //Misma estructura que los constructores pero solo con una linea
}
```

Ejemplo de uso básico:

MI COMPRENSIÓN: debemos saber diferenciar sobre los **atributos** y los **parámetros**, cuando queramos darle valores al atributo le introduciremos los valores que queramos en la sintaxis al declarar un nuevo objeto:

atributo:

```
v1= new Volumen (altura,radio);
```

parámetro: al parámetro le escribiremos las variables en la llamada al método, esto se utiliza cuando no tenemos ciertos atributos declarados en la clase.

```
total=op.dividirEnteros(num1, num2);
```

Si no tenemos atributos en la clase no hace falta agregar nada en la declaración de **objeto**

```
Operaciones op= new Operaciones();
```

RELACIONES ENTRE CLASES

A partir de ahora vamos a tener que tener una relación entre las distintas clases.

Tenemos 3 tipos:

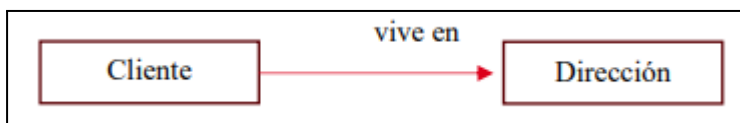
1. Asociación
2. Dependencia
3. Herencia (tema siguiente)

ASOCIACIÓN

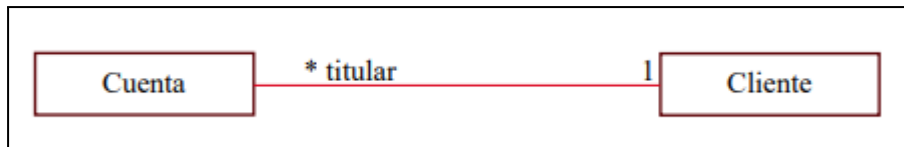
Tenemos dos subtipo de asociación: unidireccionales (solo vamos hacia una dirección), bidireccional (vamos en dos direcciones)

EJ:

Unidireccional



Bidireccionales



A las asociaciones se le suelen dar casos particulares:

- a) *Agregación*: Clase de donde parte la relación representa el TODO y las clases relacionadas representan las PARTES.

MI INTERPRETACIÓN: esto es una tipo de asociación donde una clase que forme una **entidad** (rombo, polígono, departamento) lo cual se considera **TODO** estará formado por diferentes partes (la figura del rombo está formada por segmento, el departamento por despacho...) por lo que serían las partes. En resumen son las partes las que dan forma al TODO y el todo es la representación que le damos en la realidad.

- b) *Composición*: La clase TODO controla la existencia de las clases PARTE.

MI INTERPRETACIÓN: es que las PARTES dependen de la existencia del TODO, lo cual será una entidad.

DEPENDENCIA

Es la relación que existe se expresa de forma coloquial como “Clientes y Servidor”, en este sector los clientes son el **objeto** el cual solicita una **función/servicio**, y el servidor será el **objeto** que nos permita llevar a cabo la **función**.

HERENCIAS

(Prohibidas en este tema)

IMPORTANTE: no es tan importante establecer el tipo de relación en el nombre de las clases, pero si hay que comprender que tipo de atributos van en cada clase y no crear incoherencias entre las relaciones.

Supermercado contiene producto

Supermercado (servicios) Departamentos

Supermercado (productos) proveedores

Supermercado (limpiar) Trabajadores

CONECTAR ATRIBUTOS ENTRE CLASES

Una de las principales dudas que me ha surgido es como conectar los atributos entre series, una de las respuestas es mediante el uso de los *getters* para dar valores.

```
//CREAR BILLETE  
b1.crearPrecio(saltos, m1.getPersonas());
```

En este caso vemos un método en main el cual contiene el valor de un atributo de otra clase.

¡NO RECOMENDADO!

PASO DE PARÁMETROS

Diferenciamos entre dos tipos de pasos de parámetros:

1. **Por valor:** para aquellos valores primitivos. En este caso realmente lo que estamos pasando es una copia (cogemos el argumento, se crea una copia dentro del argumento y se la pasamos)

QUÉ OCURRE CUANDO PASAMOS A UN MÉTODO UN OBJETO

Es fundamental que antes de que pasemos el objeto a un método POYO, es básico que debemos tener instanciado el objeto.

analogía → Antes de pasar un objeto por caja debes tener creado el objeto.

```
Producto p;  
LineaDeVenta lx;
```

Después una vez tengamos los principales datos del objeto lo instanciamos:

```
p=new Producto(nombre,refrigerado,precioBase);
```

Luego:

```
lv= new LíneaDeVenta (p,cantidad);
```

PASAR OBJETOS A MÉTODOS

En esta versión de pasar objetos a métodos lo haremos metiéndolo como parámetros:

```
public double calcularSubtotalV2 (double porcentaje, Producto p) {  
    subTotal=cantidad*p.calcularPVP(porcentaje);  
    return subTotal;  
}
```

Ambos métodos sirven para lo mismo, la diferencia es que tenemos que comprender si la propia clase está relacionada, ya que este será un determinante para que sea **atributo** o **parámetro**.

REPASOS DE MÉTODOS UTILIZADOS HASTA AHORA

BÁSICO:

```
public double sumar (double a, double b) {  
    return a+b  
}
```

INTERMEDIO:

```
public double comprarBilletes (Billete b, int numPersonas) {  
    return b.calcularPrecio ()*numPersonas;  
}
```

ACTUALES:

```
public (nombre objeto a devolver (Billete)) buscarBillete (int num) {  
    Bilete b2=num;  
    return b2;  
}
```

```
public (nombre objeto (Alumno) ) buscarAlumno (String dni, ALumno [] lista) {  
    Alumno a...  
    return a;  
}
```

```
}
```

PASO DE ARRAYS COMO ARGUMENTOS Y DEVOLUCIÓN DE ARRAYS

EJEMPLO BÁSICO DEL PASO A PASO DE CREACIÓN:

Declaramos el array y luego creamos los constructores correspondientes:

```
private double totalVenta;  
private String nombreVendedor;  
private double listaImpuestos [];
```

Importante: no se **inicializa el array**, los valores se le dan en el **main**.

```
public Venta(double totalVenta, String nombreVendedor, double[] listaImpuestos) {  
}  
  
public Venta() {  
}
```

ARRAY DE DOUBLES (NO OBJETOS):

```
public double calcularMediaImpuestos () {  
    double suma=0, media;  
    for (int i = 0; i < listaImpuestos.length; i++) {  
        suma+=listaImpuestos[i]; //SUMA IMPUESTOS  
    }  
  
    return suma/listaImpuestos.length; //MEDIA  
}
```

EN EL MAIN

Declaramos:

```
Venta y;
```

Pedimos datos **antes** de instanciar la clase:

(En este caso se lo damos a cascaporras pero se podría tanto leer)

```
double totalVenta=0;  
String nombreVendedor="Angel";  
double [] listaImpuestos= {30, 0, 56, 89};
```

Tras tener los datos instanciamos:

```
//PEDIMOS DATOS DE UNA VENTA:  
v= new Venta (totalVenta, nombreVendedor, listaImpuestos);
```

ARRAYS DE OBJETOS

Para instanciar un array de objetos vamos a recordar previamente como se declaran los arrays normales:

(tipo de dato) [] (nombre variable = new (tipo de dato) [(tamaño)];

Ejemplo Real → int [] lista = new int [tam];

Ahora bien, teniendo en cuenta que esta es la forma en la que llevábamos a cabo la creación de arrays vamos a ver los objetos.

(nombre de la clase) [] (nombre variable) = new (nombre de la clase) [(tamaño)];

Ejemplo Real → Vehiculo [] lista = new Vehiculo [tam];

COMPRENDER: En estos tipos de arrays, una cosa a comprender es que dentro del objeto podremos encontrar diferentes tipos de datos.

¿Cómo instanciar el objeto?:

Lo principal será pedir los datos o bien introducirlos tu mismo a cascaporras. Ahora bien una vez tenemos los datos llevaremos a cabo la siguiente sintaxis:

Al igual que en lo tradicional podemos llevarlo mediante la siguiente forma:

1. Declaramos el objeto en la clase Principal:

Vehiculo v;

2. Instanciamos el array:

Vehiculo [] lista = new Vehiculo [tam];

3. Pedimos datos y declaramos el objeto:

v = new Vehiculo ((nombre variable), (nombre variable));

4. Introducimos el objeto en el array: (la i es un ejemplo de la posición del array).

lista [(i)] = v;

Manera recomendada de crear los arrays de objetos:

1. Declarando el array y a la vez:

Vehiculo [] lista = new Vehiculo [tamaño];

2. Instanciamos Objeto:

lista [(variable)] = new Vehiculo (Atributo 1, Atributo 2...);

CÓMO BUSCAR UN OBJETO CONCRETO

Buscar por criterios

d

COMANDOS:

PROBLEMAS TÍPICOS:

Es importante tener en cuenta que el array guarda direcciones de memorias, tener cuidado al mostrar los arrays de objetos.

→ IMPORTANTE utilizar el toString en las diferentes clases ya que es lo que nos permitira solucionar el problema.

MÉTODO BUSCAR OBJETO EN ARRAY

Lo primero que vamos a tener que hacer es que cada producto tendrá que tener un atributo único que lo identifique (PK), a este número exclusivo de cada objeto aplicaremos un tipo de dato **INT**.

Ejemplo De Metodo:

```
public Producto buscarById (int id) {
    boolean encontrado=false;
    int i=0;
    while ( i < array.length && !encontrado) {
        Producto de Lista == listaProducto [i];
        if (deLista.getId() == id) {
            encontrado= true;
        }
        else
            i++;
    }
    if (encontrado) {
        return true } else { return false }
}
```

En el siguiente ejemplo se nos muestra una sintaxis básica de un método que tiene como finalidad buscar un producto en una lista.

Y si queremos modificar los precios? → Una opción recomendada para este caso es combinar el método **buscar**, con un set que llame al precio, esto nos permitirá buscar el **id** del producto para luego modificarlo.

COMO BORRAR ARRAYS EN JAVA (OPCIÓN 1)

Debemos partir de la base de que en java no se pueden borrar arrays, por lo que necesitamos crear un identificador que nos permita indicar si el producto por así decirlo existe o no.

Para ello vamos a llevar a cabo la creación de un nuevo atributo, el que por lo general será un booleano, y luego para setearlo pues le cambiaremos la condición para llevar a cabo la acción.